Qualitative and Quantitative Simulation Paradigms

Module 3 - ABE 721: Agricultural Systems Modelling and Simulation

Kadeghe Fue, PhD, P.Eng(T)

Department of Agricultural Engineering
School of Engineering and Technology
Sokoine University of Agriculture (SUA)

Lecture Outline

1. 1. Simulation Paradigms
2. 2. Numerical Integration (ODEs)
3. 3. Numerical Solutions for PDEs
4. 4. Parameter Estimation & Optimization

Simulation Execution

What happens after formulation?

In Module 2, we formulated agricultural realities into mathematical models (ODEs/PDEs). However, most real-world models are highly non-linear and lack exact analytical solutions.

Module 3 Focus:

- **Execution:** How the computer advances time and tracks variables.
- **Numerical Integration:** Translating continuous calculus into discrete algebra.
- **Parameter Estimation:** Calibrating models using real-world data.

Quantitative vs. Qualitative Simulation

Quantitative Simulation

- Relies on strict, deterministic numerical data and hard boundaries.
- Based on laws of physics (calculus, linear algebra).
- *Example:* Calculating exact pipe diameter for an irrigation network to maintain 200 kPa.

Qualitative Simulation

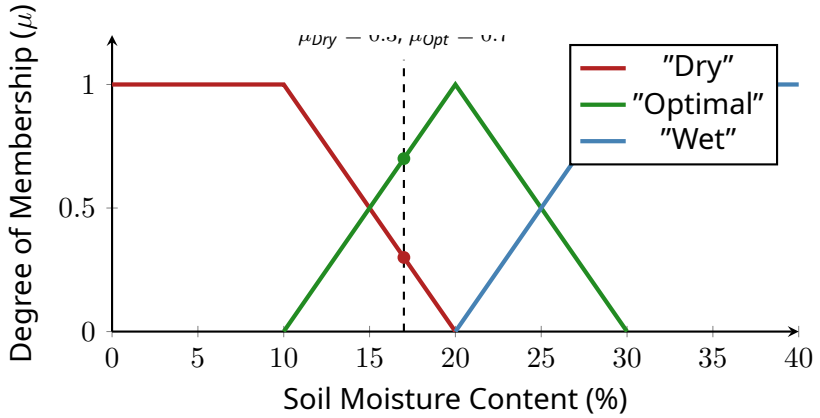
- Deals with trends, behaviors, and fuzzy boundaries.
- Handles variables difficult to measure (e.g., ”farmer satisfaction”).
- *Example:* Simulating adoption rates of a new technology based on perceived economic risk.

Soft Computing: Fuzzy Logic

Agricultural systems often require **Fuzzy Logic** (Tayfur, 2014) for qualitative simulation.

- Unlike Boolean logic (1 or 0), fuzzy logic utilizes **Membership Functions** (μ) where an element partially belongs to a set.
- Example: Soil moisture isn't just "Dry" or "Wet". It can be 30% Dry and 70% Optimal simultaneously.
- Allows for expert rules: *"IF Soil is Dry AND Temp is Hot, THEN Irrigation is High."*

Fuzzy Logic Membership Functions



Continuous vs. Discrete-Event Simulation

Beyond data types, the computational architecture is determined by the internal clock.

1. Continuous Simulation

- State variables change smoothly and continuously.
- Clock advances in tiny, uniform, rigid increments (Δt).
- Governed by ODEs/PDEs (e.g., crop dry matter accumulation).

2. Discrete-Event Simulation

- System state changes only at discrete points triggered by ”events”.
- Between events, the system state is perfectly constant.
- Clock jumps instantly to the next scheduled event, bypassing inactivity.
- Governed by Queuing Theory.

Example: Sugar Mill Receiving Queue

Tractor arrives ($t = 2.1$) $\rightarrow$ Weighing begins $\rightarrow$ Weighing ends ($t = 2.5$) $\rightarrow$ Unloading begins.

Agent-Based Modelling (ABM)

- A growing paradigm in spatial agricultural modelling.
- The system is a collection of autonomous decision-making entities (”agents”).
- Each agent (e.g., an insect or a farmer) assesses its situation and acts based on localized rules.
- *Use Case:* Simulating the spatial epidemiological spread of plant diseases or evaluating policy adoption across a region.

The Need for Numerical Integration

Continuous agricultural models rely on Initial Value Problems (IVPs):

$$\frac{dy}{dt} = f(t, y) \quad \text{with} \quad y(t_0) = y_0$$

- Complex biological/physical ODEs have **no analytical solution**.
- We must use **Numerical Integration** to step through time.
- Calculates approximate values of y at discrete steps: $t_{n+1} = t_n + h$.
- h represents the step size (Δt).

The Foundation: Taylor Series Expansion

All numerical integration methods are derived from the Taylor series. We project the future state $y(t_{n+1})$ using current derivatives:

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2!}y''(t_n) + \frac{h^3}{3!}y'''(t_n) + \cdots + O(h^p)$$

$O(h^p)$ represents the **truncation error** resulting from ignoring higher-order terms in the computer algorithm.

First-Order Methods: Forward Euler

The simplest technique. Truncates the Taylor series after the first derivative.

Euler's Formula

$$y_{n+1} = y_n + hf(t_n, y_n)$$

- **Physical Meaning:** Next state = Current state + (Slope $\times$ Time Step).
- **Global Error:** Proportional to $O(h)$. This is a "First-Order" method.
- **Limitation:** Inherently inaccurate for non-linear curves because it assumes slope is perfectly constant over the entire step.

Second-Order Methods: Heun's Method

Improves upon Euler using a Predictor-Corrector logic to average the slopes.

Step 1 (Predictor): Standard Euler prediction.

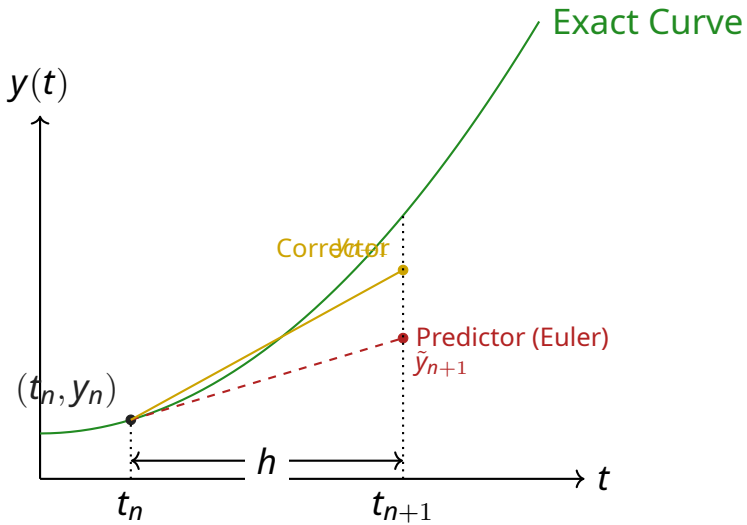
$$\tilde{y}_{n+1} = y_n + hf(t_n, y_n)$$

Step 2 (Corrector): Average the initial slope and predicted final slope.

$$y_{n+1} = y_n + \frac{h}{2}\left[f(t_n, y_n) + f(t_{n+1}, \tilde{y}_{n+1})\right]$$

Result: Global error is $O(h^2)$, a massive improvement in stability.

Geometric Interpretation of Heun's Method



Fourth-Order Runge-Kutta (RK4)

The **RK4** is the gold standard for continuous agricultural systems simulation. It evaluates the slope at multiple strategic points within h.

$$
\begin{aligned}
k_1 &= f(t_n, y_n) \\
k_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\
k_3 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\
k_4 &= f(t_n + h, y_n + hk_3)
\end{aligned}
$$

Final Weighted Average

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad \text{Error is } O(h^4)$$

Adaptive Step-Size (RK45)

- Real-world solvers (MATLAB's 'ode45', SciPy) use adaptive step-size algorithms (RK45).
- Calculates the step using both an RK4 and an RK5 method simultaneously.
- The difference between the two determines the local error.
- **Dynamic Adjustment:**
  - Shrinks h during rapid change (e.g., sudden rainfall event).
  - Expands h during dormant periods to save CPU memory.

Partial Differential Equations (PDEs)

While ODEs model variables over *time* (Lumped Models), PDEs model variables over both *time and space* (Distributed Models).

Example: 1D Heat/Diffusion Equation (Heat in greenhouse soil)

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

To solve this computationally, we must replace continuous calculus derivatives with discrete algebraic approximations using the **Finite Difference Method (FDM)**.

FDM Discretization

Using Taylor expansions on a space-time grid ($\Delta x, \Delta t$):

Temporal (Forward Difference):

$$\frac{\partial u}{\partial t} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t}$$

Spatial (Central Difference):

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2}$$

The Explicit Scheme and CFL Stability

Substituting yields the **Explicit Scheme** to find the future state:

$$u_i^{n+1} = u_i^n + \frac{\alpha \Delta t}{(\Delta x)^2} \left(u_{i+1}^n - 2u_i^n + u_{i-1}^n \right)$$

The CFL Condition

Explicit FDM suffers from severe instability if Δt is too large. Von Neumann analysis requires the Courant-Friedrichs-Lewy condition:

$$\frac{\alpha \Delta t}{(\Delta x)^2} \leq 0.5$$

The Crank-Nicolson Method

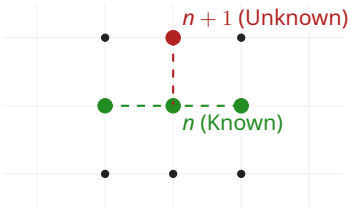
To bypass the CFL stability limit, modern models use the **Implicit Crank-Nicolson** method, averaging the spatial derivative across time steps n and $n+1$.

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{\alpha}{2}\left[\frac{\delta_x^2 u^n}{(\Delta x)^2} + \frac{\delta_x^2 u^{n+1}}{(\Delta x)^2}\right]$$

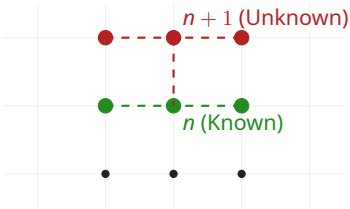
- **Unconditionally stable.**
- Requires formulating and solving a massive tridiagonal matrix system because future states depend on unknown neighboring future states.

FDM Computational Stencils

Explicit Scheme



Crank-Nicolson Scheme



Finite Element Method (FEM)

FDM fails on complex, curved 3D geometries (e.g., tractor chassis). The **Finite Element Method** is the standard for mechanical analysis.

- Divides CAD model into interconnected geometric sub-volumes (tetrahedrons) called **Finite Elements**.
- Converts PDEs into simultaneous algebraic equations using the Principle of Virtual Work.

Governing Matrix Equation

$$[\mathbf{K}]\{\mathbf{U}\} = \{\mathbf{F}\}$$

$[\mathbf{K}]$: Stiffness Matrix, $\{\mathbf{U}\}$: Nodal Displacement, $\{\mathbf{F}\}$: Force Vector.

The Parameter Estimation Problem

A perfect mathematical model (e.g., solved with RK4) is useless if its **parameters** (K_s, specific heat capacity) are incorrect.

- **Calibration / Inverse Modelling:** Computationally finding the optimal parameter set $\mathbf{p}$ so simulated outputs match historical data.
- Fundamentally an **Optimization Problem**.

Objective Function Formulation

We define a cost function representing the ”error” between simulation and reality. The universal standard is the **Sum of Squared Errors (SSE)**:

$$J(\mathbf{p}) = \sum_{i=1}^{N} [y_{obs}(t_i) - y_{sim}(t_i, \mathbf{p})]^2$$

The ultimate goal is to find the singular optimal parameter vector $\mathbf{p}^*$ that globally minimizes $J(\mathbf{p})$:

$$\mathbf{p}^* = \arg\min_{\mathbf{p}} J(\mathbf{p})$$

Gradient Descent: Levenberg-Marquardt (LMA)

For non-linear models, we use numerical gradient descent solvers. The **Levenberg-Marquardt Algorithm** is the gold standard for Non-Linear Least Squares.

$$\mathbf{p}_{k+1} = \mathbf{p}_k - \left[\mathbf{J}^T\mathbf{J} + \lambda \text{diag}(\mathbf{J}^T\mathbf{J})\right]^{-1} \mathbf{J}^T\mathbf{e}(\mathbf{p}_k)$$

- $\mathbf{J}$ is the Jacobian matrix (slope pointing downhill).
- λ is a damping factor.
- **Limitation:** Gradient methods get trapped in *local minima*, failing to find the true global optimum.

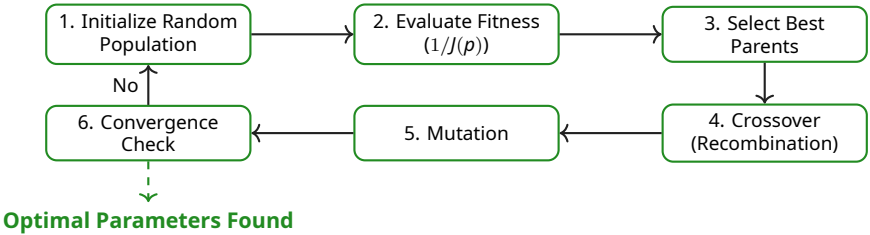
Soft Computing: Genetic Algorithms (GA)

To avoid local minima in massive agricultural models, we use Metaheuristics.

Genetic Algorithms use probabilistic natural selection, not gradients.

1. **Initialization:** Generate a random ”population” of parameter sets (Chromosomes).
2. **Fitness Evaluation:** Run model, calculate error. Fitness = $1/Error$.
3. **Selection:** Choose best parents.
4. **Crossover:** Combine parent numerical values into children.
5. **Mutation:** Randomly tweak values to explore new map areas.

The GA Evolutionary Cycle



Particle Swarm Optimization (PSO)

Mimics the social kinematics of a flock of birds. A swarm of ”particles” moves through the parameter space, updating velocity based on personal best (P_{best}) and swarm global best (G_{best}).

Velocity Update

$$\mathbf{v}_i^{(t+1)} = w\mathbf{v}_i^{(t)} + c_1 r_1(\mathbf{P}_{best,i} - \mathbf{p}_i^{(t)}) + c_2 r_2(\mathbf{G}_{best} - \mathbf{p}_i^{(t)})$$

$$\mathbf{p}_i^{(t+1)} = \mathbf{p}_i^{(t)} + \mathbf{v}_i^{(t+1)}$$

Faster convergence than GA in continuous mathematical spaces like soil hydrology.

Summary of Module 3

- **Paradigms:** Simulation depends on quantitative vs. qualitative approaches (Fuzzy Logic), and Continuous vs. Discrete-Event clocks.
- **ODE Integration:** Forward Euler provides the basis, but Runge-Kutta (RK4) is required for stability and accuracy in agriculture.
- **PDE Integration:** Finite Difference Methods (FDM) require Implicit schemes (Crank-Nicolson) to bypass strict CFL stability limits.
- **Optimization:** Calibrating models requires advanced Metaheuristics (GA, PSO) to minimize error without falling into local minima.

End of Module 3

Questions & Discussion